

J.A.R.V.I.S. — Phase 1 (v3)

Planning & Project Charter | Cognitive Architecture Edition

Phase 1 Planning	Phase 2 Requirements	Phase 3 Design	Phase 4 Implementation	Phase 5 Testing	Phase 6 Deployment
------------------	----------------------	----------------	------------------------	-----------------	--------------------

1. Project Identity

Project name	J.A.R.V.I.S. — Personal AI Agent (v1) → AI Company (v2)
Platform	Windows 10/11 (v1) + Cloud/Online (v2)
Language	Python 3.11
Developer level	Intermediate
Architecture	Modular Monolith (v1) → Extended Modular Monolith with Multi-Agent Layer (v2)
Memory model	Cognitive architecture — 8 memory systems modelled on the human brain
Project type	Personal · Solo

2. Purpose & Goals

Primary purposes	Personal productivity Developer assistant Entertainment / fun project
Vision statement	A voice-activated Windows AI agent that boosts daily productivity, assists with coding, manages the PC autonomously, and evolves into a full AI company of specialist agents — each with a human-brain-inspired cognitive architecture that accumulates knowledge, learns from experience, and improves over time through memory consolidation.
v1 goal	Single intelligent agent: voice, PC control, coding, trading, email, memory
v2 goal	Full AI company: 14 specialist agents, background operation, vision, self-learning, online dashboard, local + cloud availability

3. Scope

In scope — v1	In scope — v2	Out of scope — v1
Wake word · Voice I/O · PC control Web search · File management Email · Coding assist · Trading data Short + long term memory (Mem0) Knowledge graph (NetworkX) Episodic memory · Context budget	14 specialist agents · Multi-agent delegation · Background operation Vision + screen control · Dashboard Local + cloud · Self-learning Full cognitive architecture · Mem0 Neo4j · Memory consolidation Forgetting curve · Spreading activation	Mobile/cross-platform Smart home Multi-user Custom ML training Auto-trading GUI dashboard

4. Cognitive Architecture Vision

Critical architecture note — what 'learning' means: The LLM (Claude, GPT-4o, Mistral) never changes. Its weights are frozen and trained by its provider. What improves over time is the *context injected into every API call* — richer notes, better-organised facts, deeper domain knowledge retrieved from the memory systems below. This pattern is called RAG (Retrieval Augmented Generation). Every major AI product (Perplexity, Cursor, GitHub Copilot, NotebookLM) uses it. The LLM is the cortex — general intelligence, frozen. The memory architecture is the hippocampus — experience, accumulated over time.

J.A.R.V.I.S. implements a cognitive architecture modelled on the human brain. The human brain is not one memory system — it is eight distinct systems operating simultaneously, each with different storage mechanisms, retrieval triggers, and decay rates. This architecture replicates all eight using purpose-built AI libraries and custom components. The result is the closest any software system can come to human-like learning, recall, and knowledge accumulation without modifying the underlying LLM weights.

5. Constraints & Assumptions

Technical	Windows OS only for v1. Microphone required. Internet for LLM API calls. 16GB+ RAM recommended for v2 with local models.
Memory	Mem0 handles semantic + associative memory. Episodic, procedural, confidence, and metacognitive memory are custom modules built on top.
LLM models	Commercial APIs (Claude, GPT-4o) for critical tasks. Local models (Ollama + Mistral) for background tasks to manage cost.
Budget	Free tiers where possible. ChromaDB, NetworkX, Mem0, Whisper all free. API costs managed by routing background tasks through local models.
Learning	Agents do not modify LLM weights. All learning is RAG-based: richer context injected per call via the 8-system cognitive architecture.

6. Success Criteria

v1 done when	User says "Hey Jarvis", speaks a command, agent responds with voice and performs the action. System recalls preferences from past sessions via Mem0.
v2 done when	14 agents operate in the background, build software, design assets, conduct research, and improve their own codebase. Dashboard accessible from phone. Nightly consolidation runs. Forgetting curve keeps memory lean.